GAZİ ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BMT317 – ALGORİTMALAR

PROJE ÖDEVİ

Sıralama Algoritmalarının Performans Karşılaştırması

Öğrenci Adı Soyadı: Murat Yalçın

Öğrenci Numarası: 24181616002

Tarih: 29/10/2026

Öğretim Elemanı: Doç. Dr. Adem Tekerek

Özet

Bu rapor, yedi karşılaştırmalı sıralama algoritmasının C++17 ile gerçekleştirilmesini ve deneysel performans karşılaştırmasını sunmaktadır. Selection, Insertion, Bubble, Shell, Quick, Merge ve Heap Sort algoritmaları; rastgele, sıralı, ters sıralı ve kısmen sıralı olmak üzere dört veri dağılımı ve 100 ile 100000 arasında değişen altı veri boyutu üzerinde incelenmiştir. Her algoritma aynı başlangıç verisinin bağımsız bir kopyası üzerinde, beş ölçüm yinelemesi ve bir ısınma turu ile çalıştırılmış; çalışma süresi, karşılaştırma sayısı, veri hareketi sayısı ve yardımcı bellek gereksinimi kaydedilmiştir. Sonuçlar, ikinci dereceden karmaşıklığa sahip algoritmaların büyük veri kümelerinde $n \log n$ algoritmalara göre yaklaşık üç mertebe daha yavaş kaldığını göstermektedir. Rastgele 100000 elemanlı veride Quick Sort 11.15 ms, Merge Sort 12.48 ms ve Heap Sort 15.40 ms çalışırken Bubble Sort 18.23 s'ye ulaşmıştır. Sıralı veride Insertion ve Bubble Sort doğrusal davranış sergilemiş; log–log regresyonla elde edilen büyüme üsleri ikinci dereceden algoritmalar için $b \approx 2$, $n \log n$ algoritmalar için $b \approx 1.1$–1.2 olarak bulunmuştur. Rapor, algoritmaların güçlü ve zayıf yönlerini veri dağılımı, bellek erişimi ve sabit maliyetler bağlamında tartışmakta ve pratik seçim önerileri sunmaktadır.

Ödevin gerçekleştirilmesinde yer alan bazı konular için yapay zeka araçlarından yardım alınmıştır. Bu konular, rapor şablonunun LaTeX formatında oluşturulması, Uygulamanın kullanılabilir bir CLI aracı haline getirilmesi ve elde edilen verilerin grafiğe dökülmesi olarak sıralanabilir. Ders kapsamında öğrenilen sıralama algoritmaları bu uygulamaya elle eklenmiştir. Bunun yanı sıra ödevde beklenilen şekilde dataset oluşturulması ve kullanımı da yapay zeka aracı kullanmadan gerçekleştirilmiştir. Raporun içeriği de elle yazılmış olup, biçimlendirilmesi, grafiklerin ve tabloların eklenmesi için yapay zekadan yardım alınmıştır.

İçindekiler

Özet **i**

1 Giriş **1**

2 Teorik Bilgi **1**

2.1 Temel kavramlar 1

2.2 İkinci dereceden karmaşıklığa sahip algoritmalar 2

2.3 Shell Sort 2

2.4 $O(n \log n)$ algoritmalar 3

2.5 Karmaşıklık özeti 3

3 Algoritmaların Gerçekleştirilmesi **4**

3.1 Dil, araçlar ve genel yapı 4

3.2 Ortak ölçüm altyapısı 5

3.3 Algoritmaların gerçekleştirim ayrıntıları 5

3.4 Doğruluk kontrolü ve testler 6

4 Deneysel Çalışma **7**

4.1 Ölçüm ortamı 7

4.2 Veri kümeleri ve dağılımlar 7

4.3 Deney protokolü 8

4.4 Yeniden üretim 8

5 Performans Sonuçları **9**

5.1 Çalışma süresi 9

5.2 Karşılaştırma sayısı 13

5.3 Veri hareketi 13

5.4 Veri dağılımının etkisi 16

5.5 Ölçülen büyüme üsleri 16

6 Karşılaştırma ve Tartışma **17**

6.1 İkinci dereceden algoritmaların ölçeklenmesi 17

6.2 $O(n \log n)$ algoritmaların büyük verideki avantajı 17

6.3 Insertion Sort'un sıralı verideki başarısı 17

6.4 Quick Sort'un pivot seçimine bağlılığı 17

6.5 Merge Sort'un dağılımlara karşı kararlılığı 18

6.6 Heap Sort ile Merge Sort'un karşılaştırılması 18

6.7 Shell Sort ve gap dizisinin etkisi 18
6.8 Kuram ile deneyin uyumu 18
6.9 Pratik seçim önerileri 19
6.10 Geçerlilik ve sınırlılıklar 19

7 Sonuç **19**

8 Ekstra Ödevler **21**

Kaynakça **22**

Şekiller Listesi

1 Dört veri dağılımı için ortalama çalışma süresinin veri boyutuna göre değişimi (log–log eksenler, 5 yineleme ortalaması). 9
2 Ortalama karşılaştırma sayısının veri boyutuna göre değişimi (log–log eksenler, 5 yineleme ortalaması). 10
3 Ortalama veri hareketi sayısının veri boyutuna göre değişimi (5 yineleme ortalaması). Bir yer değiştirme üç hareket sayılmıştır; sıfır hareketli durumların görünmesi için düşey eksen simetrik logaritmik ölçektedir. . 11
4 Veri dağılımının çalışma süresine etkisi; her panel bir algoritma (log–log eksenler, 5 yineleme ortalaması). 11
5 Sıralama görselleştiricisinin çalışırken alınmış ekran görüntüsü. 21

Tablolar Listesi

1 Yedi algoritmanın kuramsal zaman karmaşıklıkları, kararlılık ve yardımcı bellek özellikleri. 4

2 Ölçüm ortamı ve deney parametreleri. 7

3 Algoritma ve veri dağılımına göre ortalama çalışma süresi (5 yineleme ortalaması, ms). 12

4 Algoritma ve veri dağılımına göre ortalama karşılaştırma sayısı (5 yineleme ortalaması). 10^5 ve üzeri değerler bilimsel gösterimle verilmiştir. 14

5 Algoritma ve veri dağılımına göre ortalama veri hareketi sayısı (5 yineleme ortalaması). Bir yer değiştirme üç hareket sayılmıştır; 10^5 ve üzeri değerler bilimsel gösterimle verilmiştir. 15

6 Kuramsal karmaşıklık, yardımcı bellek ve ölçülen büyüme üssü b. b, $n \geq 1000$ için ortalama çalışma süresinin n'ye göre log–log doğrusal regresyon eğimidir; $b \approx 2$ ikinci dereceden, $b \approx 1{,}1$ ise $n \log n$ büyümeyi gösterir. 16

1 Giriş

Bu rapor kapsamında yedi algoritma ortak bir C++17 arayüzü üzerinde gerçekleştirilmiş; rastgele, sıralı, ters sıralı ve kısmen sıralı dört veri dağılımı ile 100, 1000, 5000, 10000, 50000 ve 100000 elemanlı veri kümeleri kullanılmıştır. Her deney beş kez yinelenmiş ve ilk çalıştırmanın başlangıç maliyetlerini elemek için bir ısınma turu yapılmıştır. Ölçüt olarak çalışma süresi, karşılaştırma sayısı, veri hareketi sayısı ve yardımcı bellek gereksinimi ele alınmıştır.

Sonuçlar, algoritmaların kuramsal karmaşıklıklarıyla karşılaştırılmış ve log–log regresyon ile ampirik büyüme üsleri hesaplanmıştır.

Raporun izleyen bölümleri şu şekilde düzenlenmiştir: Bölüm 2'de algoritmaların teorik temelleri ve karmaşıklıkları, Bölüm 3'te C++ gerçekleştiriminin tasarımı ve ölçüm altyapısı, Bölüm 4'te deneysel yöntem ve ölçüm ortamı, Bölüm 5'te deneysel bulgular, Bölüm 6'da bulguların tartışılması, Bölüm 7'de ise sonuç ve öneriler yer almaktadır.

2 Teorik Bilgi

2.1 Temel kavramlar

Karşılaştırmaya dayalı sıralama algoritmaları, elemanları birbirleriyle karşılaştırarak doğru sırayı belirler. Bu sınıfta yer alan ve yalnızca ikili karşılaştırma kullanan hiçbir algoritma, en kötü durumda $\Omega(n \log n)$ sınırından daha hızlı olamaz. İkinci dereceden algoritmaların büyük veri kümelerinde neden dezavantajlı hale geldiği bu alt sınırla doğrudan ilişkilidir.

Bir algoritmanın değerlendirilmesinde kullanılan başlıca kavramlar şunlardır:

- **Yerinde çalışma (in-place):** Algoritmanın girdi dizisinin dışında yalnızca sabit miktarda ek bellek kullanmasıdır.
- **Kararlılık (stability):** Eşit anahtara sahip elemanların sıralama sonrasında başlangıçtaki göreli sıralarını korumasıdır.
- **Karşılaştırma sayısı:** Elemanlar arasında yapılan ikili karşılaştırmaların sayısıdır.
- **Veri hareketi:** Elemanların dizi içinde atanması, kaydırılması veya yer değiştirmesiyle oluşan veri taşıma işlemlerinin sayısıdır.
- **Yardımcı bellek:** Girdi dizisine ek olarak kullanılan bellektir; özyinelemeli algoritmalarda çağrı yığını da bu kapsama girer.

Bu ölçütler, yalnızca asimptotik karmaşıklığı değil, aynı zamanda sabit maliyetleri ve bellek erişim düzenini de görünür kılar. Örneğin aynı $O(n \log n)$ sınıfında yer alan iki algoritma, önbellek dostu erişim düzeni nedeniyle belirgin biçimde farklı sürelerde çalışabilir.

2.2 İkinci dereceden karmaşıklığa sahip algoritmalar

Selection Sort (Seçmeli Sıralama). Algoritma, dizinin sıralanmamış kısmındaki en küçük elemanı bulur ve onu sıralanmamış kısmın başına taşır. Bu işlem her adımda tek bir yer değiştirme gerektirdiğinden veri hareketi sayısı $O(n)$ düzeyinde kalır; ancak en küçük elemanı bulmak için her durumda $n(n-1)/2$ karşılaştırma yapılır. Bu nedenle en iyi, ortalama ve en kötü durum karmaşıklığı $O(n^2)$'dir. Algoritma yerinde çalışır ve kararlı değildir; uzak elemanların yer değiştirmesi eşit anahtarların göreli sırasını bozabilir.

Insertion Sort (Eklemeli Sıralama). Dizinin sol tarafında sıralı bir önek büyütülür; her yeni eleman, sıralı önek içinde doğru konuma yerleştirilir. Elemanlar kaydırılarak yerleştirildiğinde yalnızca bir atama ile çok sayıda eleman bir sağa kaydırılabilir. Sıralı girdide her eleman için tek bir karşılaştırma yeterlidir; dolayısıyla en iyi durum $O(n)$'dir. Ters sıralı girdide ise her eleman başa kadar kaydırılır ve karmaşıklık $O(n^2)$ olur. Algoritma kararlıdır ve $O(1)$ yardımcı bellek kullanır. Küçük ve kısmen sıralı veri kümelerinde pratikte sıkça tercih edilir.

Bubble Sort (Kabarcık Sıralama). Komşu elemanlar karşılaştırılır ve yanlış sıradaysa yer değiştirilir; her geçişte en büyük eleman dizinin sonuna "kabarcık" gibi taşınır. Erken çıkış optimizasyonu kullanıldığında, hiç yer değiştirmenin olmadığı bir geçişte algoritma durur ve sıralı girdide $O(n)$ sürede tamamlanır. Ortalama ve en kötü durum karmaşıklığı $O(n^2)$'dir. Kararlıdır ve yerinde çalışır; ancak her yer değiştirme üç veri hareketi ürettiğinden, özellikle ters sıralı girdide veri hareketi sayısı çok yüksektir.

2.3 Shell Sort

Shell Sort, Insertion Sort'un azalan artışlarla (gap) genelleştirilmiş halidir. Dizi önce belirli bir h aralığındaki elemanlar kendi aralarında sıralanır (h-sıralı hale getirilir), sonra h küçültülür ve son adımda $h = 1$ ile normal Insertion Sort uygulanır. Bu ön sıralama, son adımdaki kaydırma sayısını önemli ölçüde azaltır.

Bu projede $n/2, n/4, \ldots, 1$ biçimindeki klasik Shell gap dizisi kullanılmıştır. Karmaşıklık kullanılan gap dizisine bağlıdır; bu dizide en kötü durum $O(n^2)$ olabilmekle birlikte pratikte ikinci dereceden algoritmalardan belirgin biçimde hızlıdır. Shell Sort yerinde çalışır ve kararlı değildir. Daha iyi gap dizileri (örneğin Knuth'un $1, 4, 13, 40, \ldots$

dizisi veya Ciura dizisi) karşılaştırma sayısını daha da düşürebilir; ancak bu çalışmada tek bir dizinin sabit tutulması, algoritmalar arası karşılaştırmanın yorumlanmasını kolaylaştırmıştır.

2.4 $O(n \log n)$ algoritmalar

Quick Sort (Hızlı Sıralama). Quick Sort, bir pivot seçip diziyi pivotdan küçük ve büyük elemanlar olmak üzere ikiye böler; ardından alt dizileri özyinelemeli olarak sıralar. Ortalama durumda $O(n \log n)$, en kötü durumda ise bölümlemenin dengesiz olmasına bağlı olarak $O(n^2)$ karmaşıklığa sahiptir. Pivot seçimi bu nedenle kritiktir. Bu projede üçün ortancası (median-of-three) yöntemi kullanılmıştır. Bu yöntem kapsamında birinci, orta ve son elemanlardan ortanca olanı pivot yapılmıştır. Bu sayede, sıralı ve ters sıralı girdilerde dengeli bölümleme olasılığı artar.

Quick Sort yerinde çalışır ve kararlı değildir. Özyineleme yalnızca küçük bölüm üzerinde yapılıp büyük bölüm döngüyle işlendiğinde yığın derinliği $O(\log n)$ ile sınırlanır. Bölümleme sırasında yapılan yer değiştirmeler bitişik olmayan bellek erişimlerine yol açsa da, küçük sabitleri ve iyi önbellek davranışı nedeniyle pratikte çoğu zaman en hızlı genel amaçlı sıralama algoritmalarından biridir.

Merge Sort (Birleştirmeli Sıralama). Dizi ikiye bölünür, her parça ayrı ayrı sıralanır ve iki sıralı parça tek bir geçişte birleştirilir. Birleştirme işlemi sıralı iki listenin başlarındaki elemanları karşılaştırarak ilerler; bu nedenle toplam karmaşıklık her durumda $O(n \log n)$'dir. Algoritma kararlıdır; eşit elemanlarda sol parçadan gelen eleman tercih edilir. Ancak birleştirme için $O(n)$ boyutunda yardımcı dizi gerekir. Sıralı erişim düzeni sayesinde büyük veri kümelerinde öngörülebilir bir performans sunar.

Heap Sort (Yığın Sıralama). Dizi bir maksimum yığın (max-heap) olarak düzenlenir; kökteki en büyük eleman dizinin sonuna taşınır ve kalan bölüm yeniden yığın özelliğine getirilir. Yığın kurulumu $O(n)$, her elemanın çıkarılması ise $O(\log n)$ olduğundan toplam karmaşıklık $O(n \log n)$'dir. Algoritma yerinde çalışır ve $O(1)$ yardımcı bellek kullanır; ancak kararlı değildir. Alt dizinlere bölme işlemleri nedeniyle bellek erişimi Merge Sort'a göre daha dağınıktır, bu durum özellikle büyük dizilerde önbellek başarımını olumsuz etkileyebilir.

2.5 Karmaşıklık özeti

Tablo 1, yedi algoritmanın en iyi, ortalama ve en kötü durum zaman karmaşıklıklarını; kararlılık ve yardımcı bellek özelliklerini özetlemektedir. Buradaki değerler kuramsal

sınırlardır; gerçek süreler sabit maliyetler, veri dağılımı ve donanım özelliklerine göre değişir.

Tablo 1: Yedi algoritmanın kuramsal zaman karmaşıklıkları, kararlılık ve yardımcı bellek özellikleri.

Algoritma	En iyi	Ortalama	En kötü	Kararlı	Yardımcı bellek
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Hayır	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Evet	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Evet	$O(1)$
Shell Sort	Değişken	Değişken	$O(n^2)$	Hayır	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Hayır	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Evet	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Hayır	$O(1)$

Shell Sort'un karmaşıklığı kullanılan gap dizisine bağlıdır; bu projede $n/2, n/4, \ldots, 1$ dizisi kullanılmıştır. Quick Sort'un $O(\log n)$ yardımcı belleği, yalnızca küçük bölüm üzerinde özyineleme yapıldığında geçerlidir.

3 Algoritmaların Gerçekleştirilmesi

3.1 Dil, araçlar ve genel yapı

Proje C++17 standardı kullanılarak geliştirilmiştir. Derleme sistemi olarak Meson ve Ninja kullanılmış; kod, Windows (MSVC veya MinGW) ve POSIX sistemlerde aynı kaynaklardan derlenebilecek şekilde taşınabilir tutulmuştur. İşletim sistemine özgü başlık veya çağrı kullanılmamıştır. Süre ölçümünde `std::chrono::steady_clock` tercih edilmiştir.

Proje kaynak kodu işlevsel olarak ayrılmıştır:

- `src/core`: eleman tipi, ölçüm sayaçları, veri üretimi, `is_sorted()` kontrolü ve kronometre.
- `src/algorithms`: her algoritma için ayrı bir derleme birimi ve algoritmaları ortak bir tanımlayıcıyla sunan kayıt (registry) yapısı.
- `src/bench`: deney planı, veri kopyalama, ısınma turları, yinelemeler ve sonuçların toplanması.
- `src/cli` ve `src/io`: komut satırı ayrıştırma ve CSV yazımı.
- `tests`: algoritmaların doğruluğunu, sayaçları, ayrıştırıcıyı ve uçtan uca bir deneyi sınayan testler.

Projenin kaynak koduna `https://github.com/muratyalcin-ce/sortbench` adresinden erişilebilir.

Projede hazır sıralama fonksiyonları kullanılmamıştır. Bütün sıralama fonksiyonları elle implemente edilmiştir.

3.2 Ortak ölçüm altyapısı

Tüm algoritmalar aynı arayüzü kullanır:

```
using Value = std::int32_t;
using SortFn = void (*)(std::vector<Value>&, SortContext&);
```

`SortContext`, algoritmaya ölçüm sayaçlarını, önceden boyutlandırılmış bir yardımcı tamponu ve ileride rastgele pivot gibi varyantlar için kullanılabilecek bir rastgele sayı durumunu taşır. Yardımcı tampon, süre ölçümü başlamadan önce ayrılır, bellek ayırma maliyeti ölçülen süreye karışmaz. Yalnızca Merge Sort bu tamponu kullanır, diğer algoritmalar yerinde çalışır.

Karşılaştırma ve veri hareketi sayıları algoritmaların içinde, `Metrics` yapısı üzerinden artırılır. Bu raporda kullanılan tanımlar şunlardır:

- **Karşılaştırma:** İki eleman arasında yapılan her ikili karşılaştırma bir kez sayılır. Ölçüm yardımcıları (`less`, `less_equal`) bu sayacı otomatik artırır.
- **Veri hareketi:** Diziye yapılan her eleman ataması bir hareket sayılır. Bir yer değiştirme (swap) klasik analizle uyumlu olarak üç hareket kabul edilir: geçici değişkene alma, birinci atama ve ikinci atama.

Bu tanımlar, farklı algoritmaların aynı ölçütle karşılaştırılabilmesini sağlar. Sayaçlar örnekleme yoluyla değil, algoritmanın her karşılaştırma ve atama noktasında doğrudan artırılır, bu nedenle aynı girdi için deterministik algoritmalarda yinelemeler arasında birebir aynı değerler elde edilir.

3.3 Algoritmaların gerçekleştirim ayrıntıları

Selection Sort. Sıralanmamış bölümdeki en küçük elemanın indisi tutulur; yalnızca gerektiğinde yer değiştirme yapılır. Karşılaştırma sayısı girdiden bağımsız olarak her zaman $n(n-1)/2$'dir, bu özellik sayaç doğrulamasında referans olarak kullanılmıştır.

Insertion Sort. Anahtar bir geçici değişkende tutulur, büyük elemanlar bir sağa kaydırılır ve anahtar boşalan konuma yazılır. Kaydırma sırasında yalnızca gerçekten yapılan atamalar sayılır; anahtarın yerinde kaldığı durumlarda gereksiz yazma yapılmaz.

Bubble Sort. Her geçişte komşu elemanlar karşılaştırılır; geçiş sırasında hiç yer değiştirme olmazsa döngü sonlandırılır. Bu erken çıkış, sıralı girdide tek bir $O(n)$ geçiş sağlar.

Shell Sort. Gap dizisi $n/2$'den başlar ve her adımda ikiye bölünür. Her gap için, o aralıktaki elemanlar üzerinde Insertion Sort'un kaydırmalı varyantı uygulanır. Gap dizisi sabit tutularak deneylerin tekrarlanabilirliği korunmuştur.

Quick Sort. Pivot olarak birinci, orta ve son elemanın ortancası seçilir; pivot son konuma taşınır ve Lomuto bölümleme şeması uygulanır. İki elemanlı aralıklar ayrıca ele alınır çünkü bu durumda orta eleman birinci veya son elemanla çakışır. Özyineleme yalnızca küçük alt bölüm için yapılır, büyük alt bölüm döngüyle işlenir. Böylece en kötü durumda bile yığın derinliği $O(\log n)$ düzeyinde tutulur.

Merge Sort. Yukarıdan aşağıya (top-down) bölme yaklaşımı kullanılmıştır. Birleştirme sırasında sağ parçadan eleman alınması yalnızca `a[j]` $<$ `a[i]` koşulu sağlandığında yapılır, eşitlik durumunda sol parça tercih edilerek kararlılık korunur. Yardımcı dizi, bağlam nesnesindeki tek bir tampondur ve tüm birleştirmelerde yeniden kullanılır.

Heap Sort. Dizi, dizi tabanlı maksimum yığın olarak uygulanmıştır. Önce iç düğümler üzerinde aşağı doğru eleme (heapify/sift-down) yapılarak $O(n)$ sürede yığın kurulur, bunun ardından kök her adımda son elemanla değiştirilip yığın küçültülerek sıralama tamamlanır.

3.4 Doğruluk kontrolü ve testler

Her ölçülen çalıştırmadan sonra sonuç, projede geliştirilen `is_sorted()` fonksiyonuyla kontrol edilir. Bu fonksiyon, dizinin azalan sırada olmayan bir çift içerip içermediğini doğrusal zamanda denetler. Ek olarak sonuç, aynı girdinin projenin kendi Merge Sort gerçekleştirimiyle sıralanmış bir referans kopyasıyla karşılaştırılır, bu sayede sıralanma durumunun yanı sıra dizideki verilerde oluşan hatalar da algılanır. Bir uyuşmazlık tespit edilirse durum standart hata akışına yazılır, ilgili CSV satırı `verified = 0` olarak işaretlenir ve program sıfır dışında bir çıkış kodu döndürür.

Doğruluk testleri yedi algoritmayı dört dağılım ve 0, 1, 2, 3, 5, 8, 17, 64, 257 ve 1023 elemanlı veri kümeleri üzerinde referans sonuçla karşılaştırır. Ayrıca tüm elemanları eşit ve yalnızca iki farklı değer içeren girdiler, 100 rastgele denemeden oluşan bir fark testi (differential fuzz), sayaç beklentileri, komut satırı ayrıştırma ve uçtan uca küçük bir deney sınanır. Test paketinde toplam 1523 kontrol bulunmakta ve tamamı başarıyla geçmektedir.

4 Deneysel Çalışma

4.1 Ölçüm ortamı

Deneyler tek bir makinede, diğer yoğun iş yüklerinden bağımsız olarak gerçekleştirilmiştir. Ölçüm ortamına ilişkin bilgiler Tablo 2 içinde verilmiştir. Süreler duvar saati (wall-clock) üzerinden ölçüldüğü için işlemci frekansı, önbellek durumu ve arka plan etkinlikleri sonuçlara küçük de olsa etki edebilir; bu nedenle yorumlarda mutlak sürelerden çok algoritmalar arasındaki göreli farklar ve büyüme eğilimleri esas alınmıştır.

Tablo 2: Ölçüm ortamı ve deney parametreleri.

Öğe	Değer
İşlemci	Intel(R) Core(TM) i5-1035G1 CPU @ 1.00GHz
Çekirdek / iş parçacığı	4 / 8
Bellek (RAM)	7.6 GiB
İşletim sistemi	Artix Linux
Derleyici	c++ (GCC) 16.2.1 20260810
Dil standardı	C++17
Derleme sistemi	Meson 1.12.1 + Ninja (release modunda derlenmiş)
IDE / çalıştırma ortamı	Komut satırı
Veri boyutları	100, 1000, 5000, 10000, 50000, 100000
Dağılımlar	rastgele, sıralı, ters sıralı, kısmen sıralı
Temel tohum	42
Yineleme	5 ölçülmüş + 1 ısınma turu
Komut	`sortbench -reps 5 -warmup 1`
Çalıştırma tarihi	2026-10-24

4.2 Veri kümeleri ve dağılımlar

Deneylerde dört farklı veri dağılımı kullanılmıştır:

- **Rastgele:** Elemanlar $[0, n)$ aralığında düzgün dağılımlı olarak üretilmiştir. Bu dağılım, algoritmanın ortalama davranışını yansıtır.
- **Sıralı:** Dizi $0, 1, \ldots, n-1$ biçiminde artan sırada üretilmiştir.
- **Ters sıralı:** Dizi $n-1, \ldots, 1, 0$ biçiminde azalan sırada üretilmiştir.
- **Kısmen sıralı:** Dizi artan sırada üretildikten sonra yaklaşık $n/10$ kez rastgele iki konumdaki eleman yer değiştirilmiştir. Böylece elemanların büyük bölümü sıralı kalırken yerel düzensizlikler oluşturulur.

Veri boyutları 100, 1000, 5000, 10000, 50000 ve 100000 olarak seçilmiştir. Tüm veri kümeleri 64 bit Mersenne Twister (`std::mt19937_64`) ile üretilmiştir. Taban tohum 42'dir, her (dağılım, boyut, yineleme) üçlüsü için tohum splitmix64 ile türetilir. Böylece deney tek bir komutla birebir yeniden üretilebilir.

4.3 Deney protokolü

Adil bir karşılaştırma için her yinelemede aşağıdaki sıra izlenir:

1. İlgili dağılım ve boyut için temel veri kümesi bir kez üretilir.
2. Her algoritma için temel veri kümesinin bağımsız bir kopyası alınır. Kopyalama, süre ölçümü başlamadan önce yapılır.
3. Algoritma, yalnızca sıralama çağrısı kronometre içinde olacak şekilde çalıştırılır.
4. Sonuç `is_sorted()` ve referans Merge Sort karşılaştırmasıyla doğrulanır.
5. Süre, karşılaştırma ve veri hareketi değerleri toplanır.

Her deney beş kez yinelenir ve aritmetik ortalama alınır; ayrıca standart sapma, en küçük ve en büyük süreler CSV çıktısında saklanır. İlk çalıştırmanın sayfa hatası, önbellek ısınması ve dinamik bağlayıcı maliyetlerinden etkilenmesini önlemek için her (algoritma, dağılım, boyut) üçlüsü için bir ısınma turu yapılır ve bu turun sonucu hesaba katılmaz.

Ölçümlerde yalnızca sıralama fonksiyonunun kendisi kronometreye dahil edilir; veri üretimi, kopyalama, doğrulama ve CSV yazımı süre dışındadır. Bu tercih, algoritmaların kendi maliyetlerini öne çıkarır. Bench tarafında ayrıca $O(n^2)$ algoritmalarının çok büyük boyutlarda uzun sürmesi durumunda deneyi kısaltmaya yarayan bir `-quadratic-limit` seçeneği bulunur; bu rapordaki tüm sonuçlarda sınır kullanılmamış, yedi algoritma da 100000 elemana kadar çalıştırılmıştır.

Sonuçlar CSV biçiminde kaydedilmiş ve `assets/plots/results.csv` dosyasında arşivlenmiştir. Her satır bir algoritma, dağılım ve boyut üçlüsü için ortalama süre, standart sapma, en küçük/en büyük süre, ortalama karşılaştırma sayısı, ortalama veri hareketi ve doğrulama durumunu içerir. Tablo ve şekiller bu CSV'den `scripts/analyze_results.py` ile üretilmiştir.

4.4 Yeniden üretim

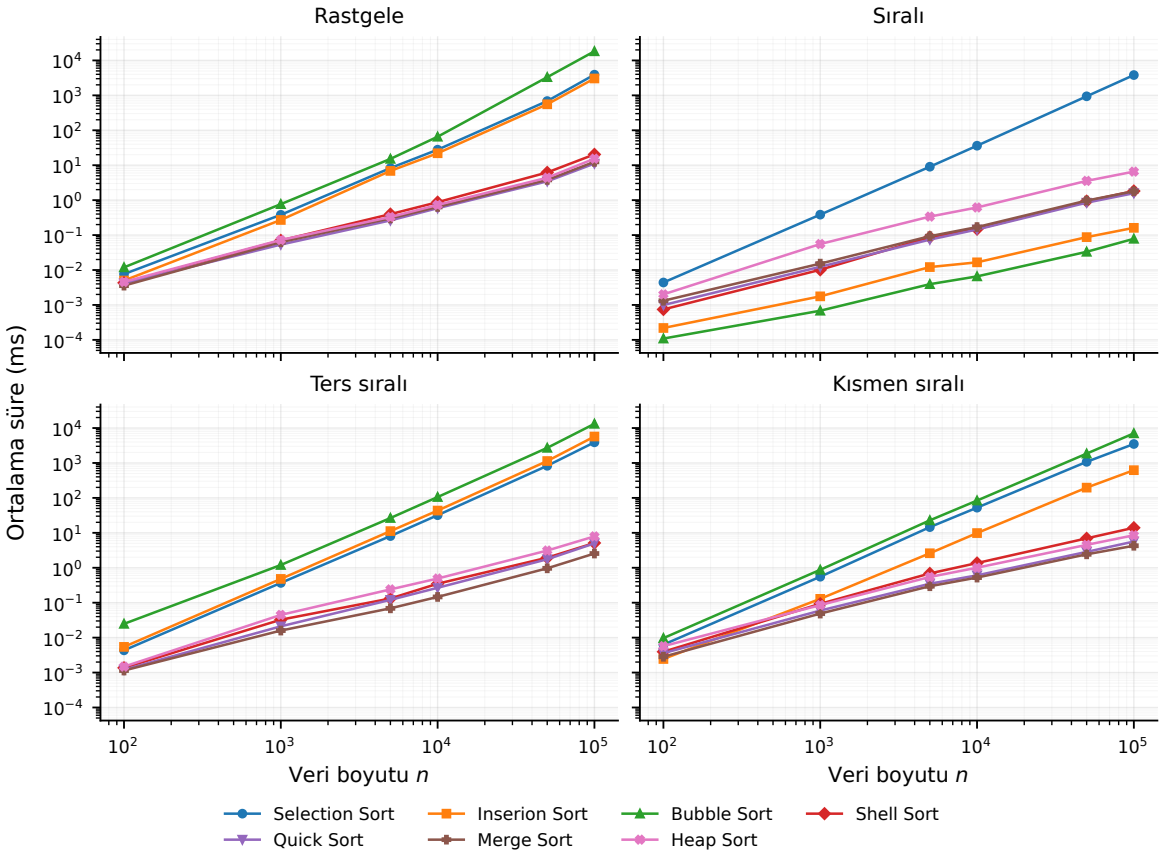
Deneyler aşağıdaki komutlarla yeniden üretilebilir:

```
meson setup build --buildtype=release
meson compile -C build
meson test -C build --print-errorlogs
./build/src/sortbench --reps 5 --warmup 1 \
    --out assets/plots/results.csv
python3 scripts/analyze_results.py \
    --csv assets/plots/results.csv --outdir assets/plots
```

`sortbench` programı varsayılan olarak yedi algoritma, dört dağılım ve altı farklı veri seti boyutu çalıştırır. Seçenekler `-help`, algoritma listesi ise `-list` ile görüntülenebilir.

5 Performans Sonuçları

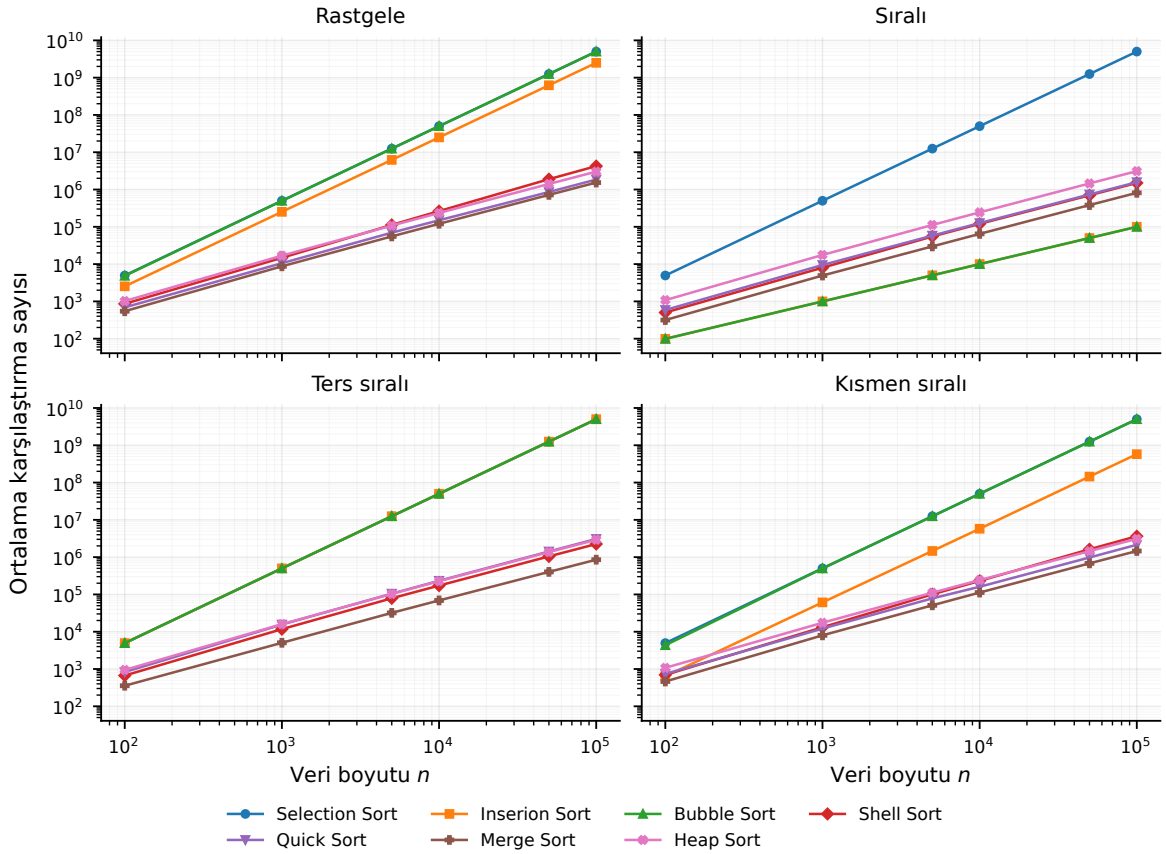
Bu bölümdeki tüm süreler beş ölçümün ortalamasıdır; karşılaştırma ve veri hareketi değerleri de aynı yinelemeler üzerinden ortalanmıştır. Ölçek farkları nedeniyle şekillerde her iki eksen de logaritmiktir.



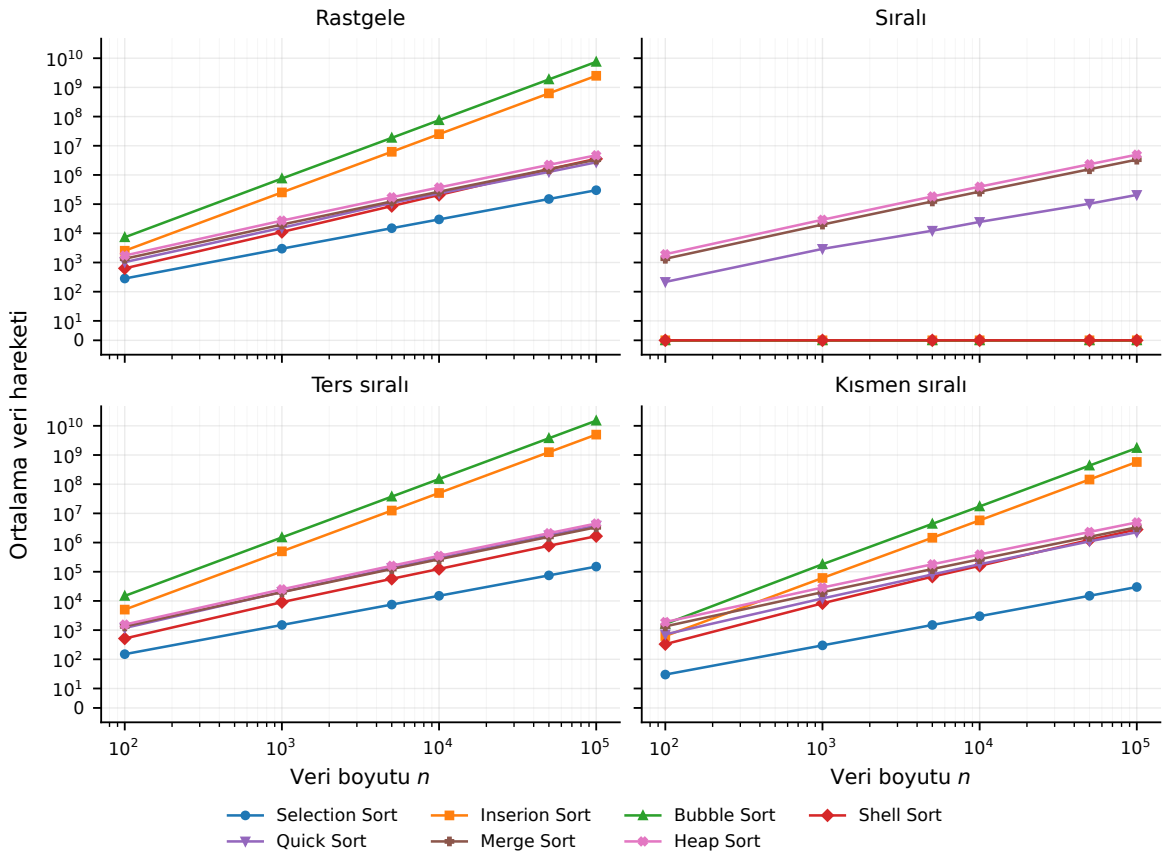
Şekil 1: Dört veri dağılımı için ortalama çalışma süresinin veri boyutuna göre değişimi (log–log eksenler, 5 yineleme ortalaması).

5.1 Çalışma süresi

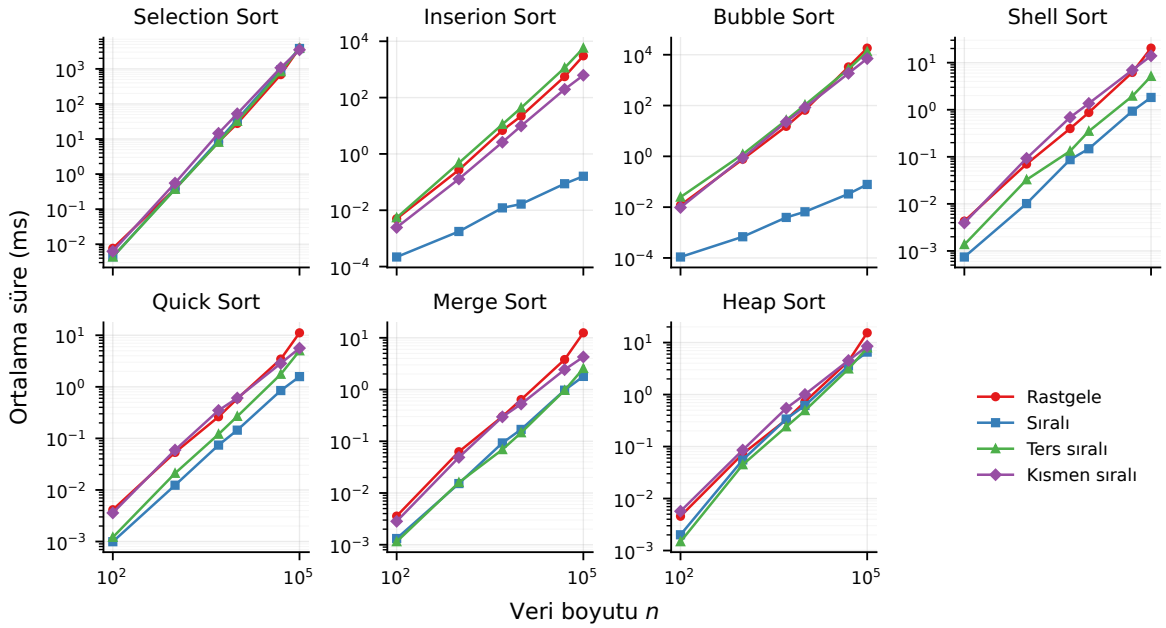
Tablo 3 ve Şekil 1, çalışma süresinin veri boyutu ve dağılımıyla değişimini göstermektedir. Rastgele veride 100000 eleman için Selection Sort 3910 ms, Insertion Sort 3042 ms ve Bubble Sort 18233 ms sürerken; Shell Sort 20.40 ms, Heap Sort 15.40 ms, Merge Sort 12.48 ms ve Quick Sort 11.15 ms düzeyinde kalmıştır. Quick Sort, aynı boyuttaki



Şekil 2: Ortalama karşılaştırma sayısının veri boyutuna göre değişimi (log–log eksenler, 5 yineleme ortalaması).



Şekil 3: Ortalama veri hareketi sayısının veri boyutuna göre değişimi (5 yineleme ortalaması). Bir yer değiştirme üç hareket sayılmıştır; sıfır hareketli durumların görünmesi için düşey eksen simetrik logaritmik ölçektedir.



Şekil 4: Veri dağılımının çalışma süresine etkisi; her panel bir algoritma (log–log eksenler, 5 yineleme ortalaması).

Bubble Sort'tan yaklaşık 1635 kat, Merge Sort'tan ise yaklaşık %10 daha hızlıdır. Bu fark, $O(n^2)$ ile $O(n \log n)$ arasındaki asimptotik ayrımın doğrudan bir sonucudur.

Tablo 3: Algoritma ve veri dağılımına göre ortalama çalışma süresi (5 yineleme ortalaması, ms).

		Veri boyutu n					
Algoritma	Dağılım	100	1 000	5 000	10 000	50 000	100 000
Selection Sort	Rastgele	0.00764	0.3804	8.29	27.82	686.6	3 910
	Sıralı	0.00439	0.3845	9.06	36.18	938.7	3 812
	Ters sıralı	0.00433	0.3684	8.08	31.97	830.5	3 899
	Kısmen sıralı	0.00629	0.5543	14.56	52.57	1 077	3 495
Inserion Sort	Rastgele	0.00500	0.2708	6.87	22.02	555.6	3 042
	Sıralı	0.00022	0.00176	0.0121	0.0166	0.0874	0.1612
	Ters sıralı	0.00543	0.4788	11.21	43.34	1 137	5 714
	Kısmen sıralı	0.00245	0.1289	2.60	9.84	196.9	621.4
Bubble Sort	Rastgele	0.0119	0.7680	15.16	65.37	3 315	18 233
	Sıralı	0.00011	0.00068	0.00393	0.00656	0.0332	0.0782
	Ters sıralı	0.0248	1.20	26.54	105.8	2 706	13 196
	Kısmen sıralı	0.00968	0.8742	22.77	83.93	1 853	7 044
Shell Sort	Rastgele	0.00430	0.0702	0.3977	0.8741	6.21	20.40
	Sıralı	0.00074	0.0102	0.0870	0.1484	0.9332	1.82
	Ters sıralı	0.00136	0.0326	0.1320	0.3490	1.94	5.12
	Kısmen sıralı	0.00394	0.0929	0.6879	1.36	6.95	13.97
Quick Sort	Rastgele	0.00413	0.0534	0.2630	0.5884	3.45	11.15
	Sıralı	0.00099	0.0124	0.0742	0.1445	0.8472	1.58
	Ters sıralı	0.00121	0.0210	0.1205	0.2672	1.75	4.99
	Kısmen sıralı	0.00359	0.0595	0.3487	0.6065	2.85	5.63
Merge Sort	Rastgele	0.00358	0.0628	0.3006	0.6393	3.80	12.48
	Sıralı	0.00132	0.0152	0.0929	0.1694	0.9722	1.79
	Ters sıralı	0.00114	0.0159	0.0690	0.1462	0.9649	2.56
	Kısmen sıralı	0.00283	0.0488	0.2962	0.5241	2.43	4.27
Heap Sort	Rastgele	0.00456	0.0724	0.3284	0.7289	4.39	15.40
	Sıralı	0.00201	0.0554	0.3380	0.6168	3.56	6.56
	Ters sıralı	0.00148	0.0447	0.2402	0.4917	3.11	7.91
	Kısmen sıralı	0.00570	0.0856	0.5450	1.01	4.52	8.52

Sıralı veride tablo belirgin biçimde değişmektedir. Selection Sort, en küçük elemanı bulmak için yine tüm karşılaştırmaları yaptığından 3812 ms'de kalır. Buna karşılık Insertion Sort 0.161 ms, Bubble Sort 0.078 ms ile neredeyse sabit maliyetli hale gelir; çünkü her eleman için tek karşılaştırma yeterlidir ve kaydırma yapılmaz. Quick Sort 1.579 ms, Merge Sort 1.794 ms, Shell Sort 1.825 ms ve Heap Sort 6.562 ms ile $O(n \log n)$ sınıfında beklenen davranışı gösterir. Quick Sort'un sıralı veride rastgele veriden daha hızlı olması, median-of-three pivotunun neredeyse dengeli bölümleme üretmesi ve yer değiştirme sayısının azalmasıyla açıklanabilir.

Ters sıralı veride Insertion Sort 5714 ms ve Bubble Sort 13196 ms'ye yükselir; her eleman başa veya sona doğru taşınır. Selection Sort 3899 ms ile boyuttan bağımsız

karşılaştırma sayısını yansıtır. Merge Sort 2.557 ms ile en hızlı sonucu verir. Quick Sort 4.995 ms, Shell Sort 5.125 ms ve Heap Sort 7.910 ms ile izler. Kısmen sıralı veride Insertion Sort 621.4 ms olarak ölçülmüştür; bu, az sayıda yerel düzensizliğin kaydırma maliyetini sınırlaması sayesinde oluşur. Bubble Sort ise 7044 ms ile hâlâ yüksektir; çünkü yerel bir düzensizlik bile çok sayıda komşu yer değiştirmeye yol açabilir.

5.2 Karşılaştırma sayısı

Tablo 4 ve Şekil 2, karşılaştırma sayılarının teorik bilgiye uyumunu göstermektedir. 100000 elemanlı rastgele veride Selection Sort ve Bubble Sort yaklaşık 5.0×10^9, Insertion Sort 2.5×10^9 karşılaştırma yapmıştır. Bu değerler $n(n-1)/2$ ve $n^2/4$ mertebeleriyle uyumludur. Aynı boyutta Shell Sort 4.24×10^6, Heap Sort 3.02×10^6, Quick Sort 1.85×10^6 ve Merge Sort 1.54×10^6 karşılaştırma ile $O(n \log n)$ davranışını doğrulamaktadır.

Sıralı veride Insertion Sort ve Bubble Sort yalnızca $n - 1 = 99999$ karşılaştırma yapar, bu, erken çıkış ve tek geçiş davranışının doğrudan göstergesidir. Selection Sort ise dağılımdan bağımsız olarak 4.99995×10^9 karşılaştırma yapmaya devam eder. Quick Sort'un karşılaştırma sayısı sıralı veride rastgele veriye göre bir miktar düşmüştür, pivot seçiminde median-of-three yöntemi kullanmanın burada avantaj sağladığı sonucuna ulaşılmıştır.

5.3 Veri hareketi

Tablo 5 ve Şekil 3, veri hareketi açısından en belirgin farkın Bubble Sort ile Selection Sort arasında olduğunu gösterir. 100000 elemanlı rastgele veride Bubble Sort 7.50×10^9 hareket üretirken Selection Sort 3.00×10^5 hareket üretmiştir. Bunun nedeni, Selection Sort'un her adımda en fazla bir yer değiştirme yapması, Bubble Sort'un ise her ters çiftte üç hareketlik bir yer değiştirme gerçekleştirmesidir. Insertion Sort 2.50×10^9 hareket ile Bubble Sort'a yakın bir yük oluşturur; ancak kaydırma tek atama ile yapıldığı için aynı karşılaştırma sayısında daha az hareket üretir.

$O(n \log n)$ algoritmalar arasında Merge Sort 3.34×10^6, Quick Sort 2.71×10^6, Shell Sort 3.56×10^6 ve Heap Sort 4.73×10^6 hareket üretmiştir. Merge Sort'un birleştirme sırasında her seviyede tampona ve geri yazması, hareket sayısını $2n \log n$ mertebesine taşır. Quick Sort'un Lomuto bölümlemesi daha az yer değiştirme üretir, Heap Sort'ta ise sift-down adımlarındaki değişimler hareket sayısını yükseltir. Sıralı veride Insertion Sort, Bubble Sort ve Selection Sort sıfır hareket üretmiştir çünkü hiçbir eleman yerinden oynamaz.

Tablo 4: Algoritma ve veri dağılımına göre ortalama karşılaştırma sayısı (5 yineleme ortalaması). 10^5 ve üzeri değerler bilimsel gösterimle verilmiştir.

		Veri boyutu n					
Algoritma	Dağılım	100	1 000	5 000	10 000	50 000	100 000
Selection Sort	Rastgele	4 950	5.00×10^5	1.25×10^7	5.00×10^7	1.25×10^9	5.00×10^9
	Sıralı	4 950	5.00×10^5	1.25×10^7	5.00×10^7	1.25×10^9	5.00×10^9
	Ters sıralı	4 950	5.00×10^5	1.25×10^7	5.00×10^7	1.25×10^9	5.00×10^9
	Kısmen sıralı	4 950	5.00×10^5	1.25×10^7	5.00×10^7	1.25×10^9	5.00×10^9
Inserion Sort	Rastgele	2 533	2.52×10^5	6.23×10^6	2.50×10^7	6.25×10^8	2.50×10^9
	Sıralı	99	999	4 999	9 999	49 999	99 999
	Ters sıralı	4 950	5.00×10^5	1.25×10^7	5.00×10^7	1.25×10^9	5.00×10^9
	Kısmen sıralı	640	61 148	1.46×10^6	5.77×10^6	1.45×10^8	5.77×10^8
Bubble Sort	Rastgele	4 848	4.97×10^5	1.25×10^7	5.00×10^7	1.25×10^9	5.00×10^9
	Sıralı	99	999	4 999	9 999	49 999	99 999
	Ters sıralı	4 950	5.00×10^5	1.25×10^7	5.00×10^7	1.25×10^9	5.00×10^9
	Kısmen sıralı	4 347	4.97×10^5	1.25×10^7	4.99×10^7	1.25×10^9	5.00×10^9
Shell Sort	Rastgele	862	14 747	1.12×10^5	2.64×10^5	1.89×10^6	4.24×10^6
	Sıralı	503	8 006	55 005	1.20×10^5	7.00×10^5	1.50×10^6
	Ters sıralı	668	11 716	78 798	1.73×10^5	1.05×10^6	2.24×10^6
	Kısmen sıralı	697	13 159	1.01×10^5	2.30×10^5	1.62×10^6	3.64×10^6
Quick Sort	Rastgele	690	10 572	69 675	1.50×10^5	8.57×10^5	1.85×10^6
	Sıralı	588	9 451	57 963	1.26×10^5	7.36×10^5	1.57×10^6
	Ters sıralı	829	15 558	1.05×10^5	2.32×10^5	1.42×10^6	3.07×10^6
	Kısmen sıralı	736	12 134	78 345	1.61×10^5	9.79×10^5	2.15×10^6
Merge Sort	Rastgele	545	8 716	55 245	1.20×10^5	7.18×10^5	1.54×10^6
	Sıralı	316	4 932	29 804	64 608	3.83×10^5	8.15×10^5
	Ters sıralı	356	5 044	32 004	69 008	4.02×10^5	8.54×10^5
	Kısmen sıralı	462	7 994	51 040	1.12×10^5	6.80×10^5	1.46×10^6
Heap Sort	Rastgele	1 023	16 847	1.08×10^5	2.35×10^5	1.41×10^6	3.02×10^6
	Sıralı	1 081	17 583	1.12×10^5	2.44×10^5	1.46×10^6	3.11×10^6
	Ters sıralı	944	15 965	1.03×10^5	2.27×10^5	1.37×10^6	2.93×10^6
	Kısmen sıralı	1 075	17 401	1.11×10^5	2.43×10^5	1.45×10^6	3.10×10^6

Tablo 5: Algoritma ve veri dağılımına göre ortalama veri hareketi sayısı (5 yineleme ortalaması). Bir yer değiştirme üç hareket sayılmıştır; 10^5 ve üzeri değerler bilimsel gösterimle verilmiştir.

		Veri boyutu n					
Algoritma	Dağılım	100	1 000	5 000	10 000	50 000	100 000
Selection Sort	Rastgele	283	2 978	14 975	29 968	1.50×10^5	3.00×10^5
	Sıralı	0	0	0	0	0	0
	Ters sıralı	150	1 500	7 500	15 000	75 000	1.50×10^5
	Kısmen sıralı	30	300	1 500	3 000	14 999	30 000
Inserion Sort	Rastgele	2 531	2.52×10^5	6.23×10^6	2.50×10^7	6.25×10^8	2.50×10^9
	Sıralı	0	0	0	0	0	0
	Ters sıralı	5 049	5.00×10^5	1.25×10^7	5.00×10^7	1.25×10^9	5.00×10^9
	Kısmen sıralı	628	61 132	1.46×10^6	5.77×10^6	1.45×10^8	5.77×10^8
Bubble Sort	Rastgele	7 315	7.54×10^5	1.87×10^7	7.49×10^7	1.87×10^9	7.50×10^9
	Sıralı	0	0	0	0	0	0
	Ters sıralı	14 850	1.50×10^6	3.75×10^7	1.50×10^8	3.75×10^9	1.50×10^{10}
	Kısmen sıralı	1 625	1.80×10^5	4.37×10^6	1.73×10^7	4.34×10^8	1.73×10^9
Shell Sort	Rastgele	630	11 114	85 867	2.06×10^5	1.57×10^6	3.56×10^6
	Sıralı	0	0	0	0	0	0
	Ters sıralı	516	9 072	57 296	1.25×10^5	7.78×10^5	1.66×10^6
	Kısmen sıralı	330	8 301	67 867	1.60×10^5	1.23×10^6	2.81×10^6
Quick Sort	Rastgele	1 028	15 430	1.10×10^5	2.31×10^5	1.27×10^6	2.71×10^6
	Sıralı	216	2 928	12 282	24 570	1.03×10^5	2.07×10^5
	Ters sıralı	1 182	20 328	1.34×10^5	2.94×10^5	1.76×10^6	3.81×10^6
	Kısmen sıralı	715	12 186	80 731	1.80×10^5	1.10×10^6	2.26×10^6
Merge Sort	Rastgele	1 344	19 952	1.24×10^5	2.67×10^5	1.57×10^6	3.34×10^6
	Sıralı	1 344	19 952	1.24×10^5	2.67×10^5	1.57×10^6	3.34×10^6
	Ters sıralı	1 344	19 952	1.24×10^5	2.67×10^5	1.57×10^6	3.34×10^6
	Kısmen sıralı	1 344	19 952	1.24×10^5	2.67×10^5	1.57×10^6	3.34×10^6
Heap Sort	Rastgele	1 732	27 211	1.71×10^5	3.73×10^5	2.21×10^6	4.72×10^6
	Sıralı	1 920	29 124	1.83×10^5	3.96×10^5	2.32×10^6	4.95×10^6
	Ters sıralı	1 548	24 948	1.60×10^5	3.50×10^5	2.10×10^6	4.49×10^6
	Kısmen sıralı	1 890	28 746	1.80×10^5	3.90×10^5	2.30×10^6	4.91×10^6

5.4 Veri dağılımının etkisi

Şekil 4, her algoritmanın dört dağılım altındaki davranışını karşılaştırmaktadır. Insertion ve Bubble Sort için dağılım etkisi çok büyüktür: sıralı veride doğrusal, ters sıralı veride ikinci dereceden davranış görülür. Quick Sort ve Merge Sort'un eğrileri birbirine daha yakındır, yine de sıralı veride daha düşük süreler elde edilir. Heap Sort orta düzeyde bir duyarlılık gösterir. Selection Sort'un eğrisi neredeyse dağılımdan bağımsızdır çünkü karşılaştırma sayısı her durumda aynıdır.

Kısmen sıralı veri, Insertion Sort için sıralı veriye göre belirgin bir maliyet artışı yaratır (0.161 ms'den 621.4 ms'ye), ancak rastgele veriye göre hâlâ çok daha hızlıdır. Bu sonuç, gerçek uygulamalarda verinin "neredeyse sıralı" olduğu durumlarda Insertion Sort'un neden makul bir seçim olduğunu açıklar.

5.5 Ölçülen büyüme üsleri

Tablo 6, kuramsal karmaşıklıkları, yardımcı bellek sınıflarını ve log–log regresyonla elde edilen büyüme üslerini bir arada vermektedir. Rastgele veri için Selection Sort $b = 1.98$, Insertion Sort $b = 2.00$ ve Bubble Sort $b = 2.21$ değerleriyle ikinci dereceden davranışı doğrulamıştır. Bubble Sort'un 2'nin üzerindeki değeri, büyük boyutlarda artan bellek trafiğinin ve önbellek kaçırmalarının bir sonucu olarak yorumlanabilir. Shell Sort $b = 1.22$, Quick Sort $b = 1.14$, Merge Sort $b = 1.13$ ve Heap Sort $b = 1.15$ ile $n \log n$ sınıfına yakın bir eğim göstermiştir.

Tablo 6: Kuramsal karmaşıklık, yardımcı bellek ve ölçülen büyüme üssü b. b, $n \geq 1000$ için ortalama çalışma süresinin n'ye göre log–log doğrusal regresyon eğimidir; $b \approx 2$ ikinci dereceden, $b \approx 1{,}1$ ise $n \log n$ büyümeyi gösterir.

Algoritma	Ortalama durum	Yardımcı bellek	b (rastgele)	b (sıralı)	Kararlı
Selection Sort	$O(n^2)$	$O(1)$	1.98	2.00	hayır
Inserion Sort	$O(n^2)$	$O(1)$	2.00	0.97	evet
Bubble Sort	$O(n^2)$	$O(1)$	2.21	1.01	evet
Shell Sort	değişken	$O(1)$	1.22	1.12	hayır
Quick Sort	$O(n \log n)$	$O(\log n)$	1.14	1.06	hayır
Merge Sort	$O(n \log n)$	$O(n)$	1.13	1.04	evet
Heap Sort	$O(n \log n)$	$O(1)$	1.15	1.04	hayır

Shell Sort $n/2, n/4, \ldots, 1$ gap dizisini; Quick Sort ise üçün ortancası (median-of-three) pivot seçimini kullanır.

Sıralı veride Insertion Sort $b = 0.97$, Bubble Sort $b = 1.01$ ile doğrusal davranış sergilerken Selection Sort $b = 2.00$ ile değişmemiştir. Quick Sort, Merge Sort ve Heap Sort için sıralı verideki üsler sırasıyla 1.06, 1.04 ve 1.04 olarak ölçülmüştür; bu algoritmaların $n \log n$ eğrisinin sıralı veride daha küçük sabitlerle gerçekleştiği görülmektedir.

6 Karşılaştırma ve Tartışma

6.1 İkinci dereceden algoritmaların ölçeklenmesi

100'den 100000 elemana çıkıldığında Selection Sort'un süresi yaklaşık 5.1×10^5 kat, Insertion Sort'un rastgele verideki süresi 6.1×10^5 kat, Bubble Sort'un süresi ise 1.5×10^6 kat artmıştır. Bu büyüme, n'nin 1000 kat artmasına karşılık sürenin yaklaşık n^2 ile ölçeklenmesinin doğrudan sonucudur. Küçük veri kümelerinde (n=100) tüm algoritmalar milisaniyenin altında kalırken, büyük kümelerde aradaki fark dakikalar mertebesine ulaşmaktadır. Bu nedenle $O(n^2)$ algoritmalar, ancak küçük veya özel yapıdaki verilerde anlamlı bir seçenek olarak değerlendirilmelidir.

6.2 $O(n \log n)$ algoritmaların büyük verideki avantajı

$O(n \log n)$ algoritmalar, her seviyede doğrusal iş yapan bölme–birleştirme veya yığın işlemlerine dayanır. 100000 elemanlı rastgele veride Quick Sort'un Bubble Sort'tan yaklaşık 1635 kat hızlı olması, asimptotik farkın pratikte ne kadar büyük olduğunu gösterir. Veri boyutu arttıkça bu fark daha da açılacaktır; çünkü $n \log n$ eğrisi ile n^2 eğrisi arasındaki oran $n/\log n$ ile büyür. Bu nedenle genel amaçlı uygulamalarda $O(n \log n)$ algoritmalar tercih edilir.

6.3 Insertion Sort'un sıralı verideki başarısı

Insertion Sort, sıralı veride her eleman için yalnızca bir karşılaştırma yapar ve hiç kaydırma yapmaz. Bu nedenle en iyi durumda $O(n)$ sürede çalışır ve 100000 elemanı 0.161 ms'de sıralar. Aynı davranış Bubble Sort'ta da görülür (0.078 ms), ancak Bubble Sort'un sabit maliyeti ve yer değiştirme eğilimi daha yüksektir. Kısmen sıralı veride Insertion Sort hâlâ avantajlıdır; çünkü her eleman yalnızca yerel düzensizliğin bulunduğu bölgeye kadar kaydırılır. Bu özellik, Insertion Sort'u küçük ve neredeyse sıralı veri kümeleri için pratik bir seçenek haline getirir.

6.4 Quick Sort'un pivot seçimine bağlılığı

Quick Sort'un performansı, bölümlemenin dengeli olup olmamasına doğrudan bağlıdır. İlk veya son elemanın pivot seçildiği klasik gerçekleştirimlerde sıralı ve ters sıralı veri, her adımda $n-1$ ve 0 elemanlı bölümler ürettiği için $O(n^2)$ en kötü duruma yol açar. Bu projede kullanılan median-of-three yöntemi, sıralı ve ters sıralı girdilerde ortanca elemanı seçerek bölümlemeyi dengeli tutar; ölçülen büyüme üssünün sıralı veride 1.06, rastgele veride 1.14 olması bunu destekler. Rastgele pivot seçimi de beklenen $O(n \log n)$ davranışı sağlar, ancak her çalıştırmada farklı bir bölümleme üreteceği için tekrarlanabilirlik açısından ek bir rastgelelik kaynağı gerektirir.

6.5 Merge Sort'un dağılımlara karşı kararlılığı

Merge Sort, girdinin dağılımından bağımsız olarak her zaman aynı bölme ve birleştirme adımlarını izler; bu nedenle en iyi, ortalama ve en kötü durumda $O(n \log n)$'dir. Ölçümlerde rastgele veride 12.48 ms, ters sıralı veride 2.557 ms, sıralı veride 1.794 ms elde edilmiştir. Aradaki fark asimptotik değil, sabit maliyetler ve önbellek davranışı kaynaklıdır: sıralı veride birleştirme sırasında karşılaştırma sayısı ve dallanma örüntüsü daha öngörülebilir hale gelir. Merge Sort'un kararlılığı ve öngörülebilir süresi, kararlılığın önemli olduğu uygulamalarda güçlü bir avantajdır.

6.6 Heap Sort ile Merge Sort'un karşılaştırılması

Büyük veri kümelerinde Merge Sort, Heap Sort'a göre belirgin biçimde daha hızlıdır: 100000 elemanlı rastgele veride 12.48 ms'ye karşılık 15.40 ms (yaklaşık %20 fark). Bunun başlıca nedeni, Merge Sort'un sıralı bellek erişiminin önbellek dostu olması, Heap Sort'un ise ağaç benzeri indis sıçramaları nedeniyle önbellek kaçırmalarına daha açık olmasıdır. Buna karşılık Heap Sort $O(1)$ yardımcı bellek kullanır, Merge Sort ise $O(n)$ boyutunda ek dizi gerektirir. Bellek kısıtı olan veya yerinde sıralama gerektiren sistemlerde Heap Sort, hız farkına rağmen tercih edilebilir.

6.7 Shell Sort ve gap dizisinin etkisi

Shell Sort, $n/2, n/4, \ldots, 1$ gap dizisiyle 100000 elemanlı rastgele veride 20,40 ms, sıralı veride 1,825 ms çalışmıştır. Ölçülen büyüme üssü rastgele veride 1.22, sıralı veride 1.12'dir; bu değerler ikinci dereceden algoritmalardan belirgin biçimde düşük, $n \log n$ algoritmalardan ise biraz yüksektir. Gap dizisi, algoritmanın karşılaştırma ve kaydırma sayısını doğrudan belirler. $n/2$ ile ikiye bölme dizisi basit ve öğreticidir; ancak Knuth'un $1, 4, 13, 40, \ldots$ dizisi veya Ciura'nın ampirik dizisi gibi alternatifler daha az karşılaştırma üretebilir. Gelecek çalışmalarda bu dizilerin aynı deney düzeneğiyle karşılaştırılması anlamlı olacaktır.

6.8 Kuram ile deneyin uyumu

Ölçülen büyüme üsleri, kuramsal beklentilerle genel olarak örtüşmektedir: Selection, Insertion ve Bubble Sort için $b \approx 2$; Quick, Merge ve Heap Sort için $b \approx 1.1$–1.2. Shell Sort için $b \approx 1.2$, gap dizisine bağlı ara bir davranışa işaret eder. Kuramdan sapmalar üç kaynaktan gelir: (i) sabit maliyetler ve küçük n'de zamanlayıcı çözünürlüğü, (ii) önbellek ve bellek bant genişliği etkileri, (iii) algoritma sabitleri ve gerçekleştirim ayrıntıları. Örneğin Bubble Sort'un rastgele veride $b = 2.21$ ölçülmesi, asimptotik sınıfın değiştiğini değil, büyük dizilerde bellek trafiğinin baskın hale geldiğini gösterir. Benzer biçimde sıralı veride Insertion Sort için $b = 0.97$ değeri, beklenen doğrusal

davranışın deneysel kanıtıdır.

6.9 Pratik seçim önerileri

- **Küçük veri kümeleri (yaklaşık $n < 50$):** Insertion Sort basit, kararlı ve düşük sabit maliyetlidir.
- **Neredeyse sıralı veri:** Insertion Sort veya Shell Sort tercih edilebilir; her ikisi de sıralılıktan belirgin biçimde yararlanır.
- **Genel amaçlı, büyük veri:** Quick Sort ortalama durumda en hızlıdır; ancak en kötü durum olasılığına karşı median-of-three veya rastgele pivot kullanılmalıdır.
- **Kararlılık gereksinimi:** Merge Sort kararlı ve öngörülebilir süre sunar; ek $O(n)$ bellek gerektirir.
- **Bellek kısıtı:** Heap Sort yerinde çalışır; hız kaybı kabul edilebilirse iyi bir alternatiftir.
- **Kaçınılması gereken:** Büyük veri kümelerinde Selection, Insertion ve Bubble Sort'un eğitimsiz kullanımı, $O(n^2)$ maliyeti nedeniyle pratik değildir.

6.10 Geçerlilik ve sınırlılıklar

Bu çalışmanın sonuçları tek bir makinede, tek bir tohum ve beş yineleme ile elde edilmiştir. Duvar saati ölçümü; işlemci frekansı, önbellek durumu ve arka plan etkinliklerinden etkilenebilir. Ölçümlerde CPU çekirdeği sabitlemesi veya frekans ölçekleme denetimi yapılmamıştır, bu nedenle küçük süre farkları istatistiksel gürültü içerebilir. Bellek kullanımı işletim sistemi düzeyinde ölçülmemiş, algoritmaların bildirdiği yardımcı bellek sınıfı ve analitik tahmin esas alınmıştır. Ayrıca veri kümeleri yalnızca 32 bit tamsayılardan oluşur, daha büyük kayıtlar veya karşılaştırma maliyeti yüksek anahtarlar farklı sabit davranışlar üretebilir. Buna karşın algoritmalar arasındaki mertebe düzeyindeki farklar ve büyüme eğilimleri bu sınırlamalardan büyük ölçüde bağımsızdır.

7 Sonuç

Bu çalışmada yedi karşılaştırmalı sıralama algoritması C++17 ile sıfırdan gerçekleştirilmiş ve dört veri dağılımı ile altı veri boyutu üzerinde deneysel olarak karşılaştırılmıştır. Ölçümler, kuramsal karmaşıklık sınıflarının pratikte belirgin biçimde gözlemlenebildiğini göstermiştir:

- $O(n^2)$ algoritmalar (Selection, Insertion, Bubble) büyük veri kümelerinde $O(n \log n)$ algoritmalara göre iki–üç mertebe daha yavaştır. 100000 elemanlı rastgele veride Bubble Sort 18.23 s'ye ulaşırken Quick Sort 11.15 ms'de kalmıştır.

- Insertion ve Bubble Sort sıralı veride doğrusal davranış gösterir, bu, en iyi durum karmaşıklıklarının deneysel doğrulamasıdır.
- Quick Sort, median-of-three pivotu ile hem rastgele hem de sıralı/ters sıralı veride $O(n \log n)$ davranışını korumuş ve genel amaçlı en hızlı algoritma olmuştur.
- Merge Sort dağılımdan en az etkilenen algoritmadır; kararlılık ve öngörülebilirlik gerektiren uygulamalar için uygundur, ancak $O(n)$ yardımcı bellek gerektirir.
- Heap Sort yerinde çalışmasına karşın bellek erişim düzeni nedeniyle Merge Sort'un gerisinde kalmıştır.
- Shell Sort, basit $n/2$ gap dizisiyle ikinci dereceden algoritmalardan belirgin biçimde hızlı, $n \log n$ algoritmalardan ise yavaştır; gap dizisi performans üzerinde doğrudan etkilidir.

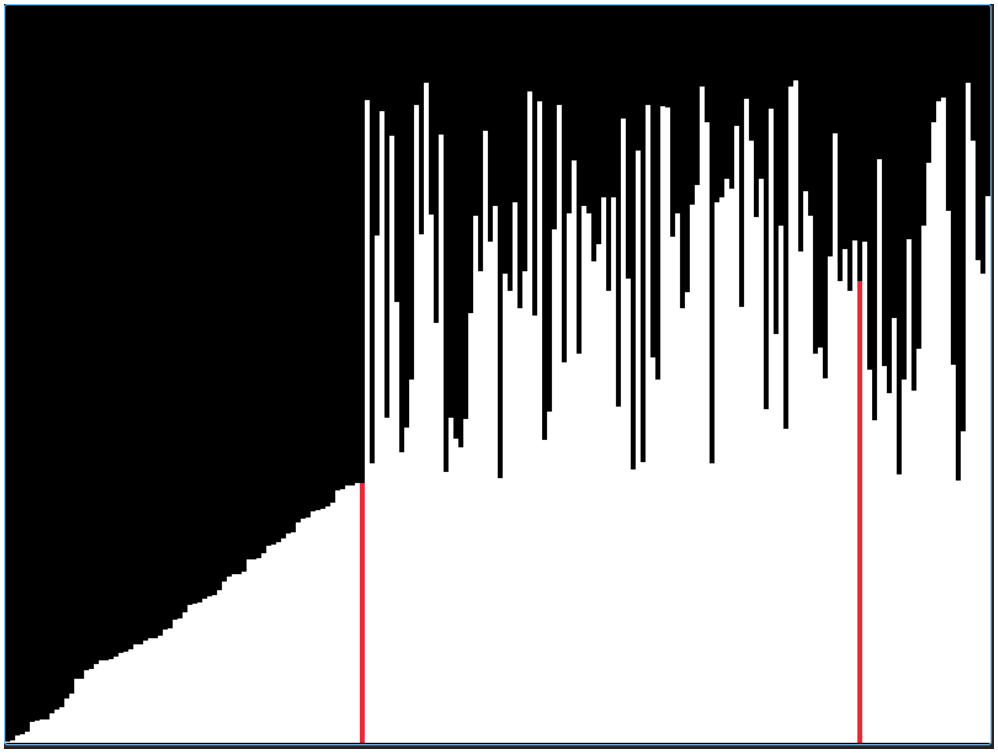
Ölçülen büyüme üsleri ($b \approx 2$ ve $b \approx 1.1$–1.2) derslerde öğretilenler ile uyumludur. Gelecek çalışmalarda farklı pivot stratejileri ve gap dizileri aynı düzenekle karşılaştırılabilir; 500000 ve üzeri boyutlar, paralel Merge Sort ve sıralama adımlarının görselleştirilmesi projenin bonus başlıkları kapsamında ele alınabilir. Ayrıca CPU çekirdeği sabitlemesi, daha fazla yineleme ve farklı donanımlar üzerinde ölçümler yapılarak sonuçların genellenebilirliği artırılabilir.

8 Ekstra Ödevler

Tesadüfen ekstra ödevler arasından sıralamaların görselleştirilmesi ile ilgili olan ödev şahsımca birkaç ay önce üzerinde uğraştığım ufak çaplı bir uygulamanın konusudur. İncelenebilmesi için GitHub'a yüklenmiştir:

`https://github.com/muratyalcin-ce/sort-visualizer`

Aşağıda, program çalıştırıldığı esnada alınan bir ekran görüntüsü (Şekil 5) eklenmiştir.



Şekil 5: Sıralama görselleştiricisinin çalışırken alınmış ekran görüntüsü.

Kaynakça

- BMT317 – Algoritmalar ders sunumları, Doç. Dr. Adem Tekerek, Gazi Üniversitesi, 2026.
- cppreference.com, *std::chrono::steady_clock*, *std::vector* ve *std::mt19937_64*, erişim: 29.10.2026, `https://en.cppreference.com/w/`.
- Meson Build System Documentation, `https://mesonbuild.com/`.
- *sortbench*, `https://github.com/muratyalcin-ce/sortbench`.
- *sort-visualizer*, `https://github.com/muratyalcin-ce/sort-visualizer`.